

Actividad 3



Desarrollo de
aplicaciones móviles

Equipo:

Zaid Castillo Hermosillo

Leonardo Pereda Chánez

Canek Lugo Soto

Carlos Sebastian Coronado

Altamirano

Práctica 7

1. Definición del Contexto

- **Temática de la aplicación:** Lista de compras y mandados organizados por categorías.
- **Objetivo de la aplicación:** El objetivo de esta práctica fue aplicar los fundamentos de Jetpack Compose creando una aplicación interactiva que integre composables personalizados, layouts adaptativos, manejo de estados y personalización de tema, consolidando los conceptos clave del desarrollo de interfaces modernas en Android.

2. Estructura Básica en Compose

- **Componentes propios creados:**
 - Vista principal contenedor de listas.
 - Tarjetas individuales para la categorización de elementos.
- **Organización y Layouts utilizados:**
 - **Column:** para la distribución vertical del encabezado y contenido principal.
 - **Row:** para el desplazamiento horizontal de las distintas tarjetas.
 - **Card:** para agrupar visualmente los elementos de cada categoría.

```
72 @Composable
73 fun EjemploColumn(modifier: Modifier = Modifier) {
74
75     Row(
76         modifier = modifier
77             .fillMaxWidth()
78             .horizontalScroll(rememberScrollState()),
79         horizontalArrangement = Arrangement.spacedBy(10.dp)
80     ) {
81
82         Card(
83             modifier = Modifier
84                 .fillMaxHeight()
85                 .width(350.dp),
86             colors = CardDefaults.cardColors(
87                 containerColor = Color(0xFFC3C9C9)
88             )
89         ) {
90             Column(
91                 modifier = Modifier.padding(16.dp),
92                 verticalArrangement = Arrangement.spacedBy(8.dp),
93             ) {
94                 Text(
95                     text = "Lista de mandado",
96                     fontSize = 25.sp,
97                     fontWeight = FontWeight.Bold,
98                     color = Color(0xFFC3B0E)
99                 )
100
101                 Text("Tomates")
102                 Text("Toallas")
103                 Text("Pasta de dientes")
104                 Text("Shampoo")
105             }
106         }
107     }
108 }
```

3. Componentes Interactivos y Manejo de Estado

- **Componentes integrados:**
 - **Campo de texto (TextField):** para el ingreso de nuevos artículos.
 - **Botón (Button):** para confirmar la adición de elementos a las listas.
- **Gestión de Estado:**
 - Control del estado mediante la memoria reactiva de Compose para actualizar dinámicamente el contenido en pantalla cada vez que se agrega o modifica una tarea/artículo.

4. Personalización del Tema y Adaptación de la Interfaz

- **Compatibilidad de colores y tema:**
 - Legibilidad y visibilidad garantizada tanto en modo claro como en modo oscuro.
- **Adaptabilidad visual :**
 - Ajustes de espaciado, extensión de pantalla , bordes redondeados y desplazamiento continuo .

```
private val DarkColorScheme = darkColorScheme(
    primary = Purple80,
    secondary = PurpleGrey80,
    tertiary = Pink80
)

private val LightColorScheme = lightColorScheme(
    primary = Purple40,
    secondary = PurpleGrey40,
    tertiary = Pink40

    /* Other default colors to override
    background = Color(0xFFFFF8FE),
    surface = Color(0xFFFFF8FE),
    onPrimary = Color.White,
    onSecondary = Color.White,
    onTertiary = Color.White,
    onBackground = Color(0xFF1C1B1F),
    onSurface = Color(0xFF1C1B1F),
    */
)

@Composable
fun ListadeMandadoTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    // Dynamic color is available on Android 12+
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else dynamicLightColorScheme(context)
        }
        else -> {
            darkTheme -> DarkColorScheme
            else -> LightColorScheme
        }
    }

    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography,
        content = content
    )
}
```

5. Prueba y Documentación

Ejecución en Dispositivo / Emulador

La aplicación fue ejecutada exitosamente en el entorno de pruebas, verificando la renderización fluida de los elementos visuales y el correcto desplazamiento horizontal entre las listas.



Actividad 3

¿Qué hace la aplicación?

Es una aplicación para registrar y gestionar eventos o tareas pendientes. Permite ver la lista de eventos, desplegar cada tarjeta para consultar detalles, registrar nuevos eventos con validación de campos y eliminar elementos mediante gestos táctiles.

Video demostrativo:

<https://www.youtube.com/shorts/q5S3AWGp1-4>

Diseño en figma:

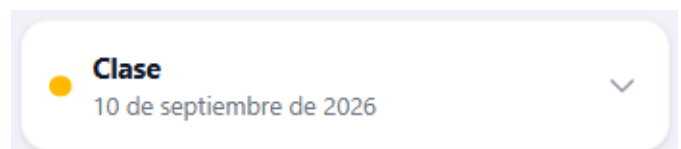
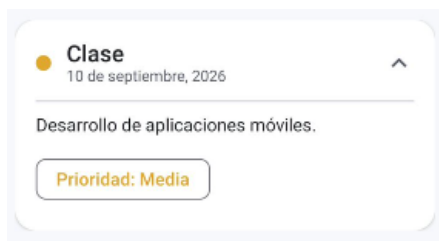
<https://www.figma.com/community/file/1677814741993229748/disenio-de-app>

Configuración Inicial

- **Entorno:** Creado en Android Studio usando Compose.
- **Estructura de Paquetes:**
 - **ui:** Interfaz visual (pantallas y componentes).
 - **model:** Estructura de los datos del evento (título, fecha, prioridad, descripción).
 - **navigation:** Manejo de cambios de pantalla.
 - **theme:** Colores y estilos.
- **EventViewModel:** Es el encargado de guardar la lista de eventos en memoria y hacer que la pantalla se actualice en tiempo real cuando agregas o eliminas un evento (mutableStateListOf).

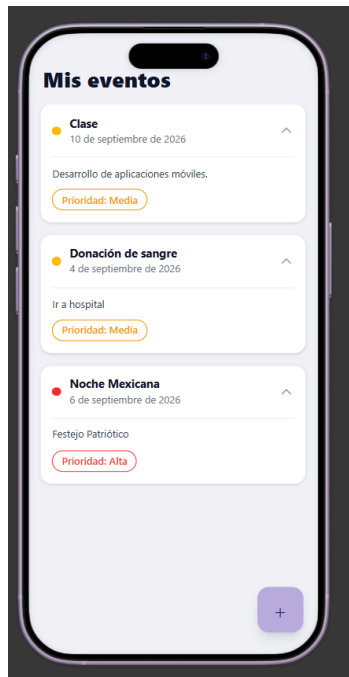
Diseño de Componentes: Tarjeta Plegable (EventCard)

- **Funcionamiento:** Cada evento se muestra en una tarjeta interactiva.
- **Plegado/Desplegado:** Al tocar la tarjeta o el botón de flecha, la tarjeta cambia de tamaño suavemente y muestra la descripción completa y la prioridad. Al volver a tocarla, se contrae.



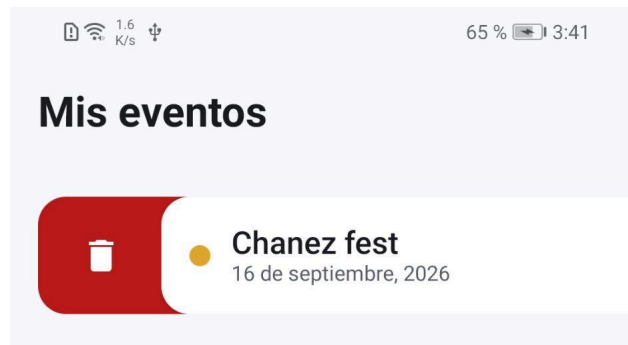
Navegación y Estados

- **Navegación :** Conecta la pantalla principal de la Lista de Eventos con la pantalla del Formulario.
- **Inyección de Dependencias:** Permite que ambas pantallas compartan la misma lista de eventos. Al guardar un evento en el formulario, aparece inmediatamente al volver a la lista



Animaciones y Gestos

- **Deslizamiento para Eliminar :** Permite borrar un evento de la lista simplemente deslizándolo con el dedo hacia la izquierda o derecha, cambiando el fondo a rojo para confirmar el borrado.
- **Animación al Agregar Eventos:** Cuando entra un nuevo evento a la lista, no aparece bruscamente, sino con un efecto de movimiento suave.



Formulario con Validaciones

- **Validaciones en tiempo real:** En la pantalla de formulario se comprueba que el título esté lleno.
- **Mensaje de Error:** Si el usuario intenta guardar con el título vacío, la app muestra un texto en color rojo

Título

El título es obligatorio

Fecha

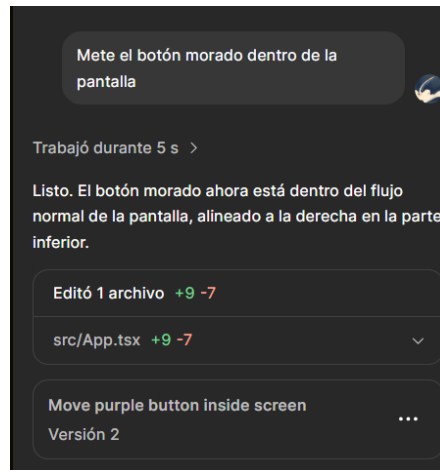
Selecciona una fecha

Pruebas y Ajustes

- **Múltiples Dispositivos y Tamaños:** Se adaptó el diseño para que la lista se ajuste a diferentes pantallas y el formulario sea desplazable cuando se abre el teclado.
- **Accesibilidad con TalkBack:** Se agregaron descripciones habladas (contentDescription) en íconos y botones, además de tamaños adecuados para facilitar el uso a personas con discapacidad visual.

Uso de la Inteligencia Artificial

- **Proporciones:** Se utilizó la herramienta de IA de figma para ajustar las proporciones adecuadas para el prototipo



- Ayudas en la redaccion y ortografia.
- Manejo de estados dentro del IDE Android Studio.